



Reinforcement Learning

Иерархическое обучение и другие
подходы к тренировке агентов



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим "+", если все хорошо
"-", если есть проблемы



Тема вебинара

Reinforcement Learning

Иерархическое обучение

Андрей Канашов

Team Lead Data Scientist в ПИК



- Ценообразование и тарификация
- Рекомендательные системы
- Прогнозирование ключевых метрик
- Анализ клиентского поведения

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе TG



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

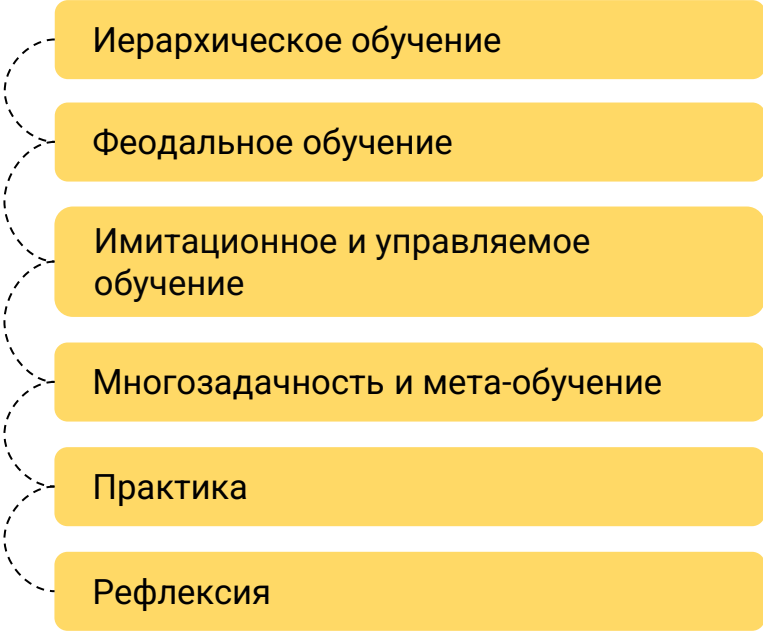


Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара



Иерархическое обучение

Феодальное обучение

Имитационное и управляемое обучение

Многозадачность и мета-обучение

Практика

Рефлексия

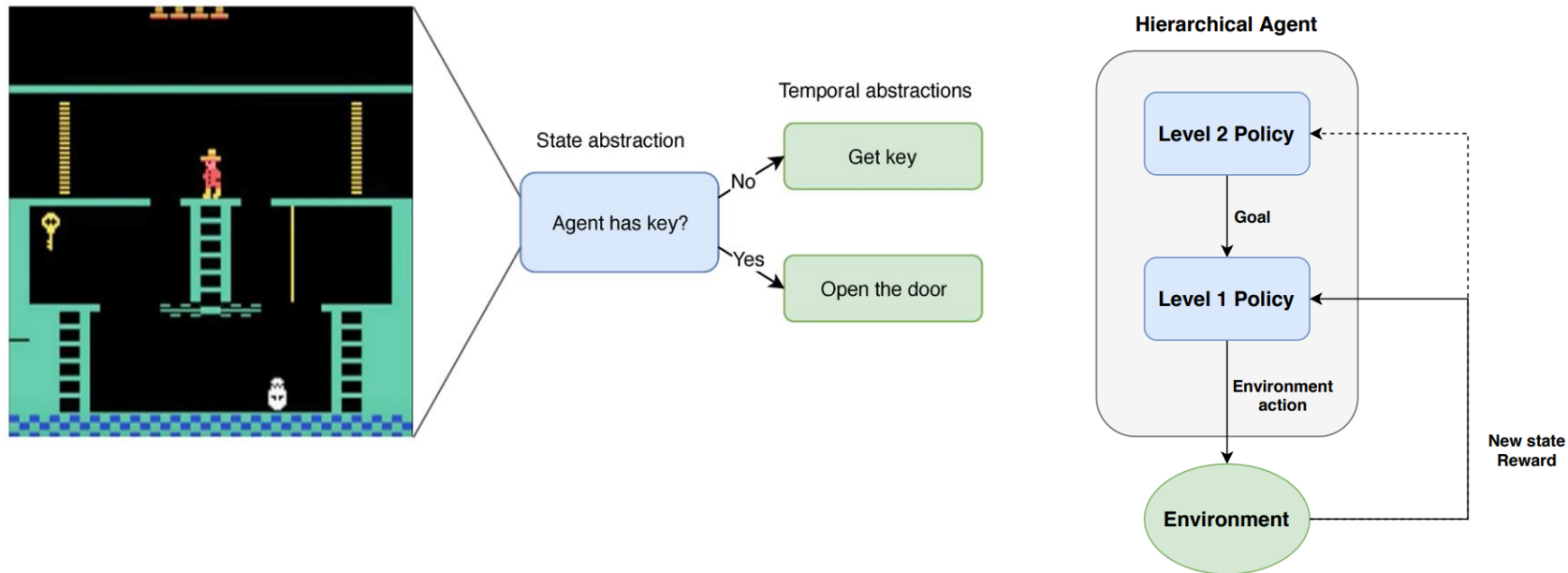
Цели вебинара

К концу занятия вы узнаете

1. Что такое иерархическое обучение и как его применять
2. О существовании ещё одного типа MDP - Semi-MPD
3. Какие ещё существуют подходы к построению процесса обучения агентов

Иерархическое обучение

Декомпозиция цели на подзадачи



Многоуровневые стратегии

Менеджер

$\pi^*(g \mid s)$ - высокоуровневая мастер-стратегия

Рабочий

$\pi(a \mid s, g)$ - низкоуровневая стратегия, работающая с простейшими действиями

Навык (option) / Цель

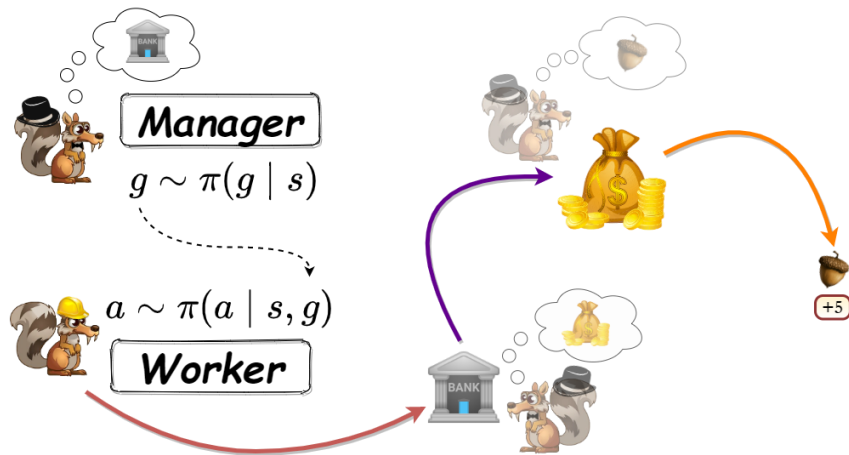
π_g - стратегия для исходного MDP

$\beta_g: \mathcal{S} \rightarrow [0, 1]$ - стратегия терминальности

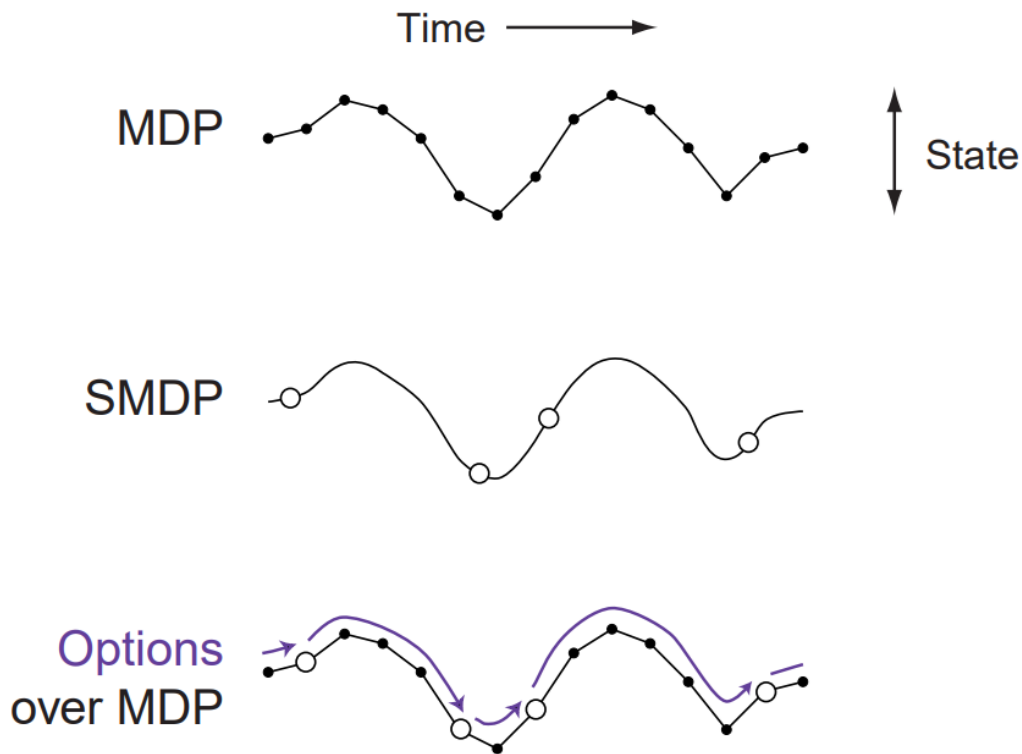
$I_g \subseteq \mathcal{S}$ - подмножество $\mathcal{S}$, где навык применим

Порождаемая траектория

$\mathcal{T} := (g_0, a_0, r_0, s_1, \text{done}_1, \beta_1, g_1, a_1, r_1, s_2, \text{done}_2, \beta_2 \dots)$



Semi-MDP



Определение

MDP называется полумарковским, если его функция перехода $p(s', \tau | s, a)$ помимо следующего состояния возвращает время $\tau > 0$, «затраченное» на выполнение данного шага.

Пространство действий

$$g \in \mathcal{G}$$

Награда

$$\bar{r}(s, a) := \sum_{t=0}^{\tau} \gamma^t r_t$$

Феодальное обучение

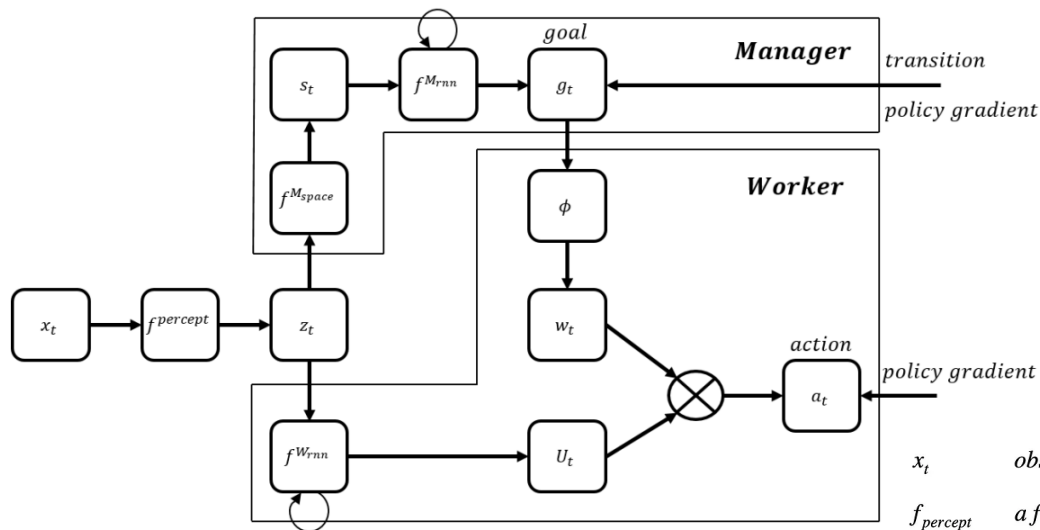
Принципы феодализма

1. Стратегия, действующая на некотором уровне абстрактности, не должна задумываться о более высокоуровневых задачах - «единственная задача вассала — выполнение задач, поставленных феодалом»

2. Высокоуровневая стратегия не рассматривает низкоуровневые действия - «феодала не заботит, как вассал будет достигать поставленных ему целей»

3. При наличии нескольких уровней иерархии действует принцип «вассал моего вассала не мой вассал»

FeUdal networks (FuN)



x_t observations from environment shared by both manager/worker

$f_{percept}$ a function that encodes the observations as internal representation (z_t) of environment

$f_{M_{space}}$ a function that outputs the latent state (s_t) for manager

$f_{M_{rnn}}$ recurrent neural network that outputs a new goal (g_t)

$f_{W_{rnn}}$ recurrent neural network that outputs a matrix (U_t)

ϕ a linear transform mapping goal into an embedding vector (w_t)

a_t the worker's outputted action after the matrix multiplication $U_t w_t$

Имитационное и управляемое обучение

Когда нужно имитационное обучение?

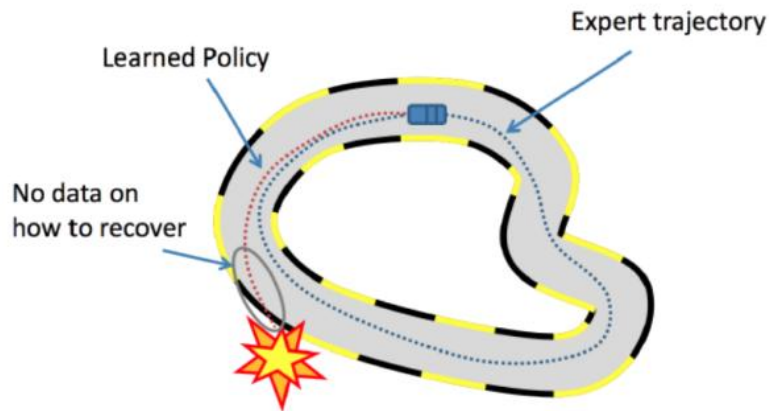
Нет функции вознаграждения
и сложно её описать

$$G_t = \sum_{r=0}^{T-1} R_{t+1+r}$$

Проще продемонстрировать
правильное поведение



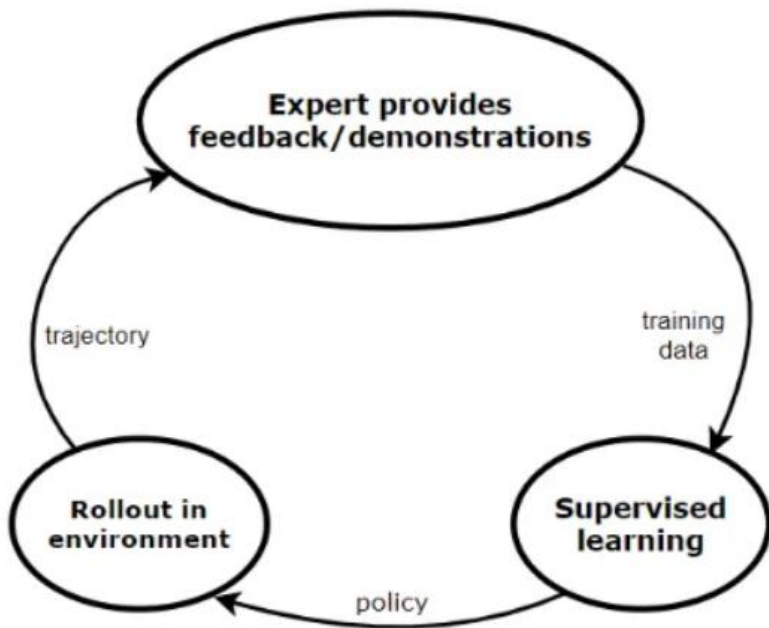
Клонирование поведения



Процесс обучения

1. Collect demonstrations (τ^* trajectories) from expert
2. Treat the demonstrations as i.i.d. state-action pairs: $(s_0^*, a_0^*), (s_1^*, a_1^*), \dots$
3. Learn π_θ policy using supervised learning by minimizing the loss function
$$L(a^*, \pi_\theta(s))$$

Прямое обучение политике



Процесс обучения

Initial predictor: π_0

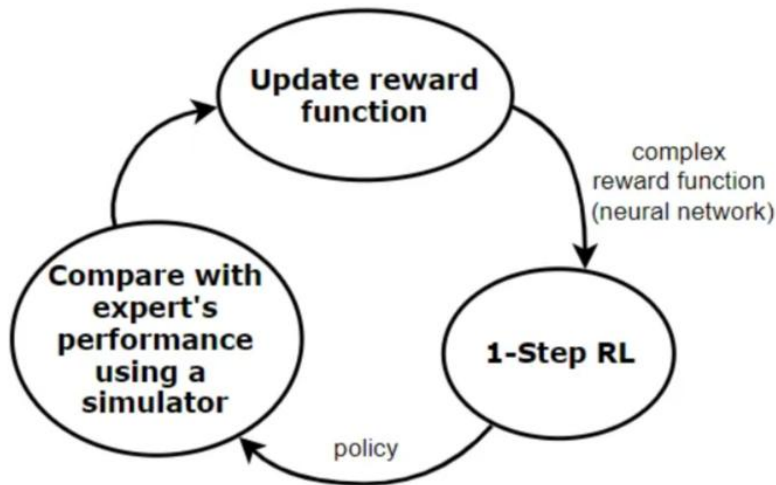
For $m = 1$:

- Collect trajectories τ by rolling out π_{m-1}
- Estimate state distribution P_m using $s \in \tau$
- Collect interactive feedback $\{\pi^*(s) \mid s \in \tau\}$
- Data Aggregation (e.g. Dagger)
 - Train π_m on $P_1 \cup \dots \cup P_m$
- Policy Aggregation (e.g. SEARN & SMILe)
 - Train π'_m on P_m
 - $\pi_m = \beta \pi'_m + (1 - \beta) \pi_{m-1}$

Обратное обучение с подкреплением

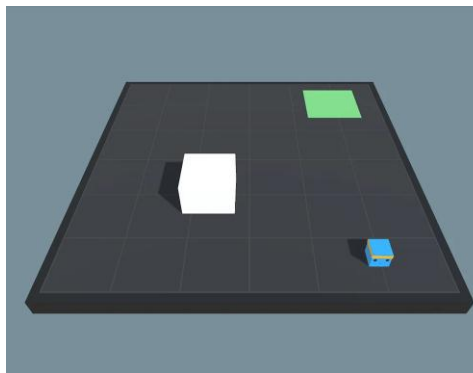
Процесс обучения

- Collect expert demonstrations: $D = \{\tau_1, \tau_2, \dots, \tau_m\}$
- In a loop:
 - Learn reward function: $r_\theta(s_t, a_t)$
 - Given the reward function r_θ , learn π policy using RL
 - Compare π with π^* (expert's policy)
 - STOP if π is satisfactory

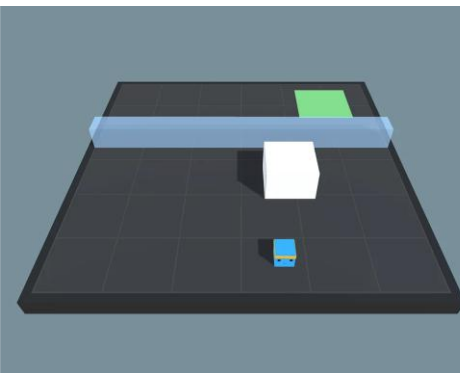


Управляемое обучение

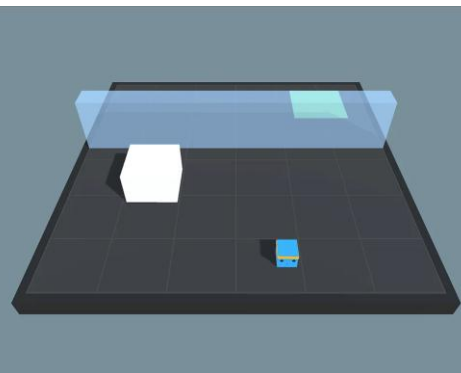
Задача 1



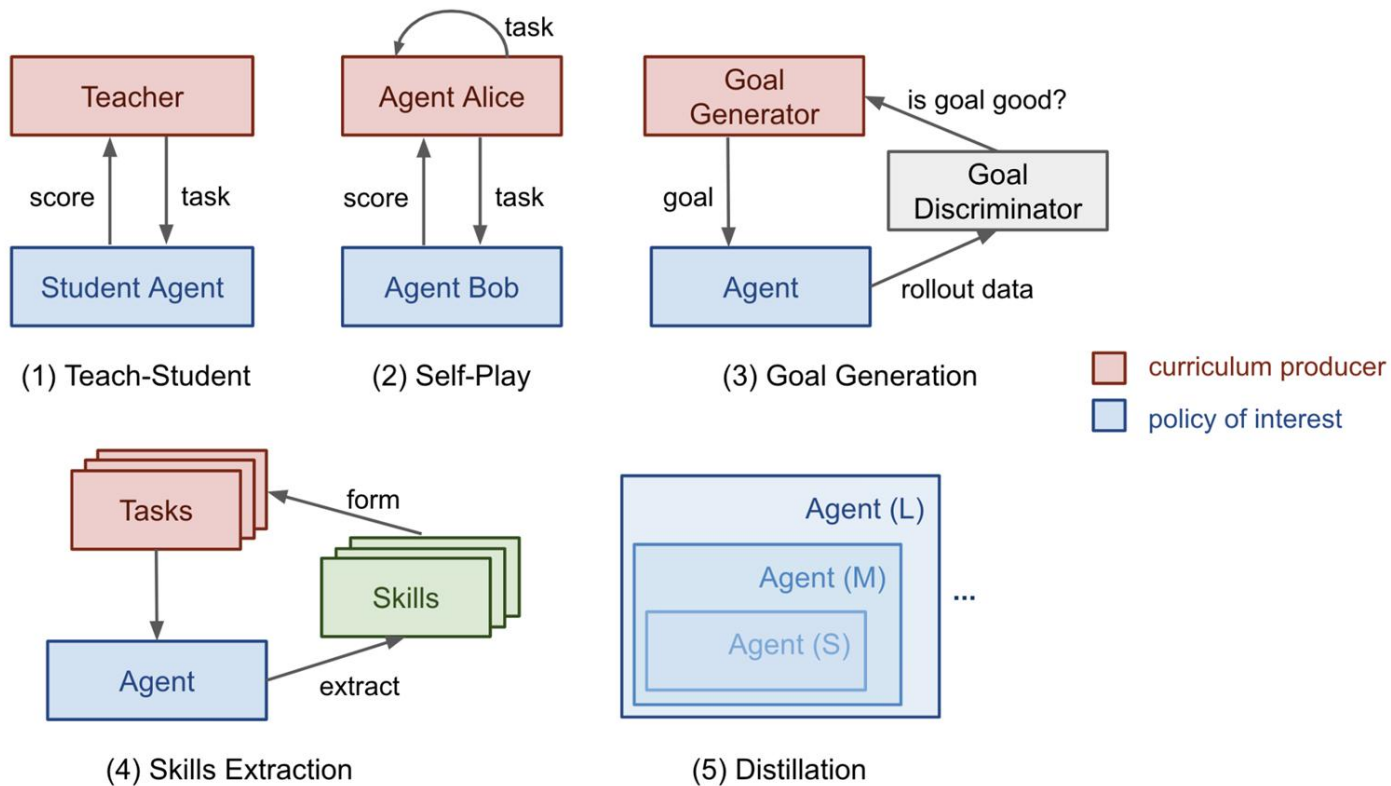
Задача 2



Задача 3



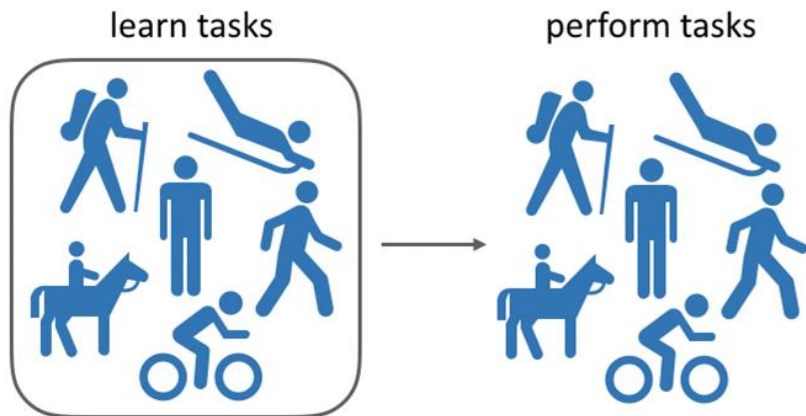
Разновидности управляемого обучения



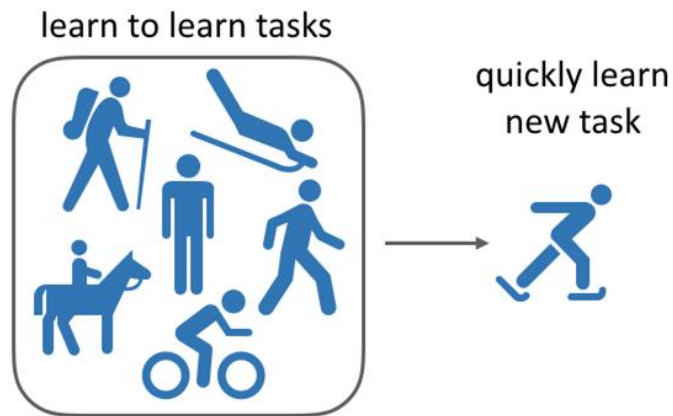
Многозадачность и мета-обучение

Сравнение

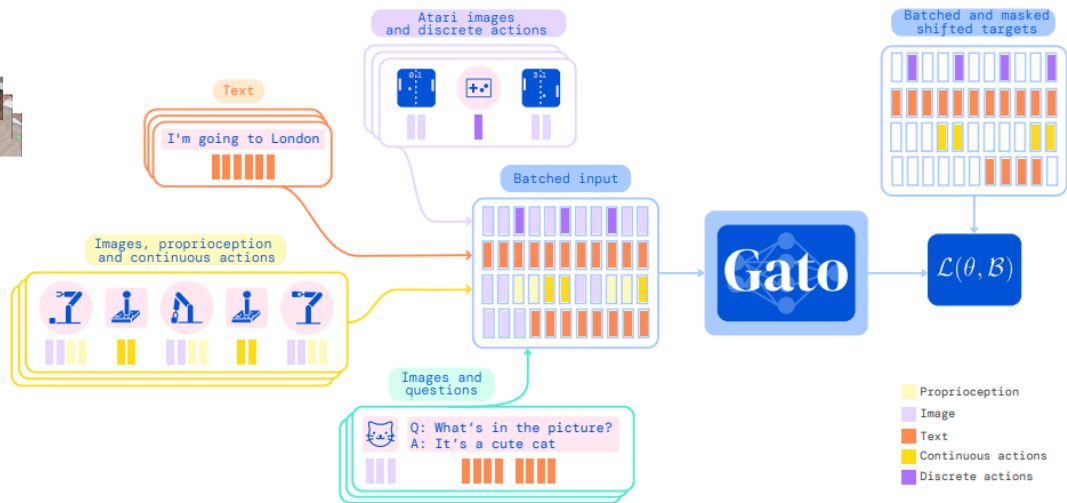
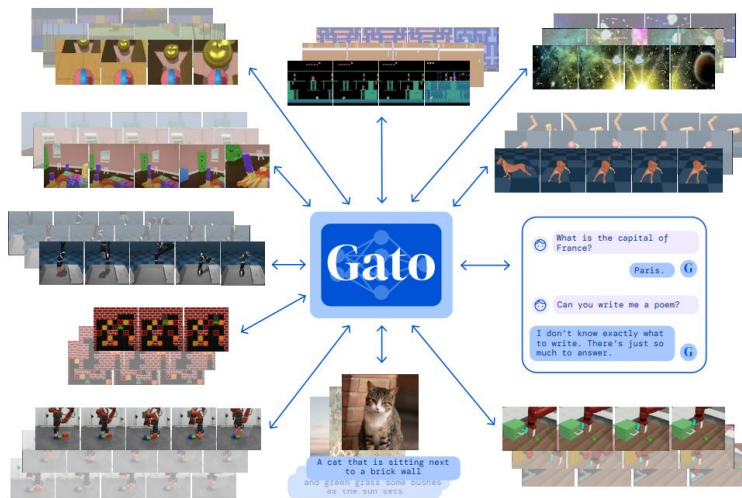
Multi-task RL



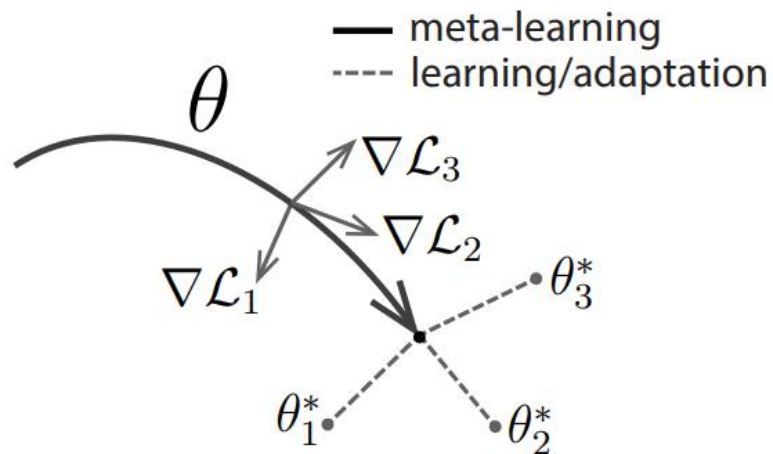
Meta RL



Multi-task RL: GaTo



Meta RL: MAML



Algorithm 3 MAML for Reinforcement Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

- 1: randomly initialize θ
 - 2: **while** not done **do**
 - 3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
 - 4: **for all** $\mathcal{T}_i$ **do**
 - 5: Sample K trajectories $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{a}_1, \dots, \mathbf{x}_H)\}$ using f_θ in $\mathcal{T}_i$
 - 6: Evaluate $\nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$ using $\mathcal{D}$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation 4
 - 7: Compute adapted parameters with gradient descent:
 $\theta'_i = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$
 - 8: Sample trajectories $\mathcal{D}'_i = \{(\mathbf{x}_1, \mathbf{a}_1, \dots, \mathbf{x}_H)\}$ using $f_{\theta'_i}$ in $\mathcal{T}_i$
 - 9: **end for**
 - 10: Update $\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$ using each $\mathcal{D}'_i$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation 4
 - 11: **end while**
-

Практика

Домашнее задание (опциональное)

1. Разобраться с кодом из практической части занятия
2. Реализовать методы для сохранения и загрузки состояния в классе DDPGAgent



Сроки выполнения: до конца курса

Рефлексия

Ключевые тезисы

Подведем итоги

1. Иерархическое обучение даёт возможность декомпозировать сложные проблемы на отдельные подзадачи и выучивать отдельные навыки для их решения, которые можно переиспользовать
2. На текущий момент нет достаточно стабильных алгоритмов позволяющих в процессе обучения самостоятельно выделять подцели и формировать навыки для их достижения
3. Имитационное обучение позволяет задавать неплохую начальную инициализацию алгоритмов или выучивать функцию наград на основе демонстраций
4. Многозадачность агентов, перенос обучения, мета-обучение - шаги к повышению обобщающей способности агентов над задачами

Список материалов для изучения

1. Статья: [FeUdal Networks](#)
2. Статья: [HIRO](#)
3. Статья: [Successor Options: An Option Discovery Framework for Reinforcement Learning](#)
4. Статья: [Generalist Agent \(Gato\)](#)
5. Статья: [Model-Agnostic Meta-Learning \(MAML\)](#)
6. Статья: [Curriculum for Reinforcement Learning](#)
7. Статья: [Goal-Conditioned Reinforcement Learning with Imagined Subgoals](#)
8. Статья: [Automatic Goal Generation for Reinforcement Learning Agents](#)
9. Статья: [Autotelic Agents with Intrinsically Motivated Goal-Conditioned Reinforcement Learning](#)
10. Статья: [Hierarchical Programmatic Reinforcement Learning via Learning to Compose Programs](#)

Следующий вебинар



21 мая 2024

Многоагентное обучение и кооперация агентов



Ссылка на вебинар
будет в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный
материал обозначен
красной лентой

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!